SPLITBLEE

Split. Share. Enjoy.

Full-Stack Product Requirements & Implementation Guide

Ocean-blue • Minimal • Mobile-first • Real-time

Design source of truth: the approved five-screen ocean-blue mockup. Preserve its airy spacing, pale-blue surfaces, deep-navy typography, restrained blue actions, simple icons and minimal illustration. Do not replace it with generic glassmorphism, neon gradients, floating-card overload or a crowded dashboard.

Build-ready product specification. Any sample names, balances, photos or expenses shown in the mockup are placeholders; production must use real authenticated user data.

Approved SplitBlee UI Mockup

Five-screen visual reference • Ocean blue • Minimal, mobile-first design



Screen sequence

1. Login • 2. Dashboard • 3. Friends Map • 4. Split Details • 5. Settings

Implementation rule: reproduce this screen hierarchy, spacing, pale-blue surfaces, deep-navy typography and restrained blue actions. Keep secondary tools in focused detail views and Settings rather than crowding the main screens.

1. Product overview

SplitBlee records shared costs, calculates fair shares, tracks who has paid, and helps groups settle balances. Use cases include restaurant bills, shopping, travel, household costs, events and custom expenses.

Core requirements

• Create groups and one-off splits; add participants, items, taxes, service charges, tips, extra amounts and discounts.

• Support equal, percentage, exact-amount and item-based splitting, with totals that reconcile exactly.

• Store friend name, photo, address, age and gender (male/female only), with consent and privacy controls.

• Track paid, partially paid, unpaid and overpaid states from recorded payment entries.

• Show monthly spending, paid amounts, outstanding amounts, net balances and real-time settlement progress.

• Provide PDF, TXT and CSV exports in Settings → Data & Export.

• Allow configurable themes, app language, default currency, regional formatting and personal display preferences.

Recommended implementation stack

Layer	Implementation
Android	Native Kotlin, Jetpack Compose, Material 3 customized to match the reference, Navigation Compose, MVVM and clean architecture
Backend	Firebase Authentication, Cloud Firestore, Cloud Storage, Cloud Functions and Firebase Cloud Messaging.
Authentication	Google sign-in and email/password, verification, password reset, sign-out and account deletion.
Maps	Google Maps SDK; request location permission only when map/location features are opened.
Exports	Generate PDF/TXT/CSV on device from authorized records; Android share sheet and Save As.
Localization	Android string resources, ICU/CLDR formatting, per-user language preference.
Testing	Kotlin coroutines/Flow, Firebase Emulator Suite, unit and integration tests.

2. Approved mockup: five-screen implementation

Match the approved mockup's screen hierarchy, spacing, proportions and visual restraint. Treat the mockup as the visual source of truth; secondary functionality belongs in detail pages, sheets or Settings.

Screen 1 — Login

• Keep the calm ocean/boat illustration, SplitBlee wordmark and “Split. Share. Enjoy.” tagline.

• Primary: Continue with Google. Secondary: Continue with Email. Optional guest preview must be isolated and clearly labelled.

• Email flow includes create account, password visibility, forgot password, verification and clear validation errors.

• Include Terms of Service and Privacy Policy. Never show fake logged-in users or balances.

Screen 2 — Dashboard

• Greeting and avatar; prominent Total Balance summary with owed-to-you, you-owe and monthly change.

• Quick actions: New Split, Groups, History and Map.

• Recent Splits list: title, date, participant count, total and Paid/Partially Paid/Unpaid status.

• Bottom navigation: Home, Groups, central New Split, Map and Profile. Keep detailed analytics in History, not on the home screen.

Screen 3 — Map and friends

• Map/List switch, searchable friends and privacy-aware profile pins.

• Tapping a friend opens a compact detail sheet: photo, name, age, gender, shared address/place, group balances and profile action.

• Location is private by default. Require explicit, revocable sharing; display last-updated time and sharing status.

• List view must work if location permission is denied. Friend profile fields are optional where appropriate.

Screen 4 — Split details

• Show expense title, optional receipt, date/time, place, creator and participant avatars.

• Bill breakdown: subtotal, item amounts, taxes, service charge, tip, extra amount, discount and final total.

• Support equal, percentage, exact and item-based splits. Show each person's share, paid amount, remaining amount and status.

• Actions: Edit (permission-controlled), Record Payment, Mark as Unpaid, Mark as Paid, add note and Share Details.

• A reminder or opening a payment app does not count as payment. Only a confirmed/manual ledger entry updates status.

Screen 5 — Settings

• Profile/account: edit name, photo, age, gender, optional address, email and connected providers.

• Appearance: Ocean Blue default; optional Mint, Lavender, Sand, Graphite and Rose; light/dark/system modes.

• Language: English default, Nepali and Hindi at launch; add more through localization resources.

• Currency and regional format selector; each expense retains its own original currency.

• Data & Export: PDF, TXT and CSV, with date-range/group/expense scope selection. Keep exports here, not on the dashboard.

• Notifications, privacy, blocked users, help, terms, privacy policy, data download and account deletion.

3. Visual design system

Element	Direction
Palette	Pale cool-blue/near-white backgrounds, deep navy text, medium-blue actions, subtle grey-blue dividers.
Typography	Clean sans-serif; clear heading hierarchy, readable secondary labels and aligned monetary figures.
Surfaces	White/light-blue surfaces with restrained elevation; no excessive glass blur or stacked floating cards.
Shape	Consistent moderate corner radii and native touch targets; avoid turning every control into a pill.
Spacing	8dp base spacing scale, consistent screen gutters, generous whitespace and predictable alignment.
Icons	Simple, consistent outline icons; no emoji as functional icons.
Responsive	Compose layouts must adapt to small and large Android screens, safe insets, font scaling and long translations.

4. Currency support

Use ISO 4217 codes and locale-aware currency formatting. Provide a searchable selector with localized names, symbols and correct minor-unit precision. Never hard-code a symbol or assume all currencies use two decimals.

Currency	Code	Example
Nepalese Rupee	NPR	■■ 1,250.00
Indian Rupee	INR	■1,250.00
Pakistani Rupee	PKR	Rs 1,250.00
US Dollar	USD	$1,250.00

Currency	Code	Example
Euro	EUR	€1,250.00
British Pound	GBP	£1,250.00
UAE Dirham	AED	■.■ 1,250.00
Saudi Riyal	SAR	■.■ 1,250.00
Qatari Riyal	QAR	■.■ 1,250.00
Kuwaiti Dinar	KWD	■.■ 1,250.000
Australian Dollar	AUD	A$1,250.00
Canadian Dollar	CAD	C$1,250.00
Japanese Yen	JPY	¥1,250
Bangladeshi Taka	BDT	■1,250.00
Sri Lankan Rupee	LKR	Rs 1,250.00

• Suggest a default from device region, but let users choose. NPR can be suggested for a Nepal-region device; never force it.

• Every expense stores its original currency and original amounts. Changing the user's default currency must not rewrite historical transactions.

• For cross-currency summaries, store base/quote currency, rate, provider and timestamp. Label converted totals as estimates.

• Use integer minor units or BigDecimal, never binary floating-point for money.

• A group selects a settlement currency. Other currencies are display conversions only unless the group explicitly creates a converted settlement.

Rounding: calculate shares in minor units and distribute any remainder deterministically so all participant shares sum exactly to the final bill total.

5. Languages and localization

Language	Locale	Requirement
English	en	Default language and fallback.
Nepali	ne	Devanagari support, translated interface and local formatting.
Hindi	hi	Devanagari support, translated interface and local formatting.
Arabic	ar	Optional expansion; requires right-to-left layout.
Urdu	ur	Optional expansion; requires right-to-left layout.
Bengali	bn	Optional expansion.
French / Spanish	fr / es	Optional expansion.

• Change language in Settings and apply immediately without logout.

• Persist language per account and locally for pre-login screens.

• Use correct plural rules, dates, digits and currency formatting. Do not auto-translate names, notes or expense titles.

• Test font rendering, wrapping and accessibility on compact screens in Nepali and Hindi.

6. Expense, split and payment logic

Expense fields

• Expense ID, group ID, creator ID, title, category, notes, place, receipt, created/updated timestamps and status.

• Currency code, subtotal, item lines, taxes[], service charges[], tip, extra amount, discount and final total.

• Participants, split method, assigned share, paid amount, remaining amount and per-person payment status.

• Support multiple named taxes/fees and fixed or percentage discounts. Validate inputs and prevent invalid totals.

Payment states

State	Meaning
Unpaid	No payment recorded for this participant's share.
Partially paid	Recorded payments are below the assigned share.
Paid	Recorded payments equal the assigned share.
Overpaid	Recorded payments exceed the share; show credit/refund handling.
Cancelled	Cancelled expense; retain an audit record.

Payments should be immutable ledger entries: amount, currency, payer, recipient, method/reference, timestamp and recorder. Corrections create reversal/adjustment entries instead of silently deleting history.

7. Monthly tracking and exports

• Monthly overview: total expenses, user's paid amount, unpaid amount, amounts owed to the user, amounts owed by the user and net balance.

• Filters: month, group, category, participant, currency and payment status.

• Show a simple monthly trend and settlement progress calculated from recorded payments divided by assigned shares.

• Exclude cancelled expenses and label partial settlements. Use a consistent account timezone policy.

Export	Contents
PDF	Readable report with date range, totals, expense breakdown, participants, shares, payment states, taxes, discounts and extra amounts
TXT	Plain-text summary and line-by-line expense/payment details.
CSV	Spreadsheet-ready rows with ISO date, IDs, group, participant, currency code, amounts and status.

• All export actions live in Settings → Data & Export.

• Scope: all records, selected month, date range, group or single expense; preview scope and record count first.

• Use UTF-8 for TXT/CSV and correctly quote CSV fields. Include currency code with every monetary value.

• Use Android share sheet or Save As. Exclude private friend details/location unless authorized and explicitly selected.

8. Firebase backend and data model

Firebase Authentication handles identity. Cloud Firestore stores profiles, groups, expenses and payment ledger entries. Cloud Storage stores profile photos and receipts. Cloud Functions enforce financial invariants, authorization-sensitive actions, notifications and audit events.

Collection	Core fields / purpose
users/{uid}	displayName, email, photoUrl, age?, gender?, address?, preferredLanguage, defaultCurrency, theme, timestamps

Collection	Core fields / purpose
groups/{groupId}	name, ownerId, memberIds, settlementCurrency, timestamps
friendships/{id}	requesterId, recipientId, status, createdAt, acceptedAt; visible only to involved users
expenses/{expenseId}	groupId, creatorId, title, category, currency, subtotal, taxes[], extras[], discount, total, splitMethod, participantShares[], statu
payments/{paymentId}	expenseId, groupId, payerId, payeeId, amount, currency, method, reference?, createdAt, createdBy
receipts/{receiptId}	expenseId, storagePath, contentType, uploadedBy, createdAt
activity/{activityId}	groupId, actorId, eventType, targetId, createdAt; append-only activity feed
exchangeRates/{pair_date}	baseCurrency, quoteCurrency, rate, provider, fetchedAt; cached reference only

Security

• Firestore Rules verify authentication, membership and ownership for every read/write.

• Friend details and addresses are visible only to approved connections and permitted group members.

• Never store passwords in Firestore. Use Firebase Auth recovery and provider linking.

• Validate totals, allocations and payment transitions in trusted Cloud Functions/transactions; clients cannot forge balances.

• Restrict Storage by owner/group membership; validate MIME type and size. Use App Check, rate limits and audit logs.

• Support account deletion and data export with clear retention rules for shared financial records.

9. Real-time, offline and notifications

• Use Firestore snapshot listeners for groups, expenses, participants and payment updates.

• Show honest syncing/offline indicators. Queue safe edits and reconcile after reconnecting.

• Use transactions/server validation for payment records and split recalculation; use idempotency keys to prevent duplicate submissions.

• Notify group members on expense creation/updates, recorded payments and reminders. Respect notification preferences.

• No fake live activity, sample balances or simulated progress in production. Empty states should invite users to create a real split.

10. Main user flows

Create a split

• New Split → title/category/place/date → group or participants → subtotal/taxes/extras/discount → split method → share preview → save.

• Backend validates total and allocation, writes expense and shares atomically, then notifies members.

Record payment

• Open expense → choose participant → enter amount/date/method/reference → confirm.

• Backend validates permissions and currency, creates ledger entry, recalculates remaining amount/status and updates the UI in real time.

Add a friend

• Search safe identifier or invite link → send request → recipient accepts → profile visibility follows privacy settings.

• Adding a friend must not automatically share their live location.

11. Suggested Android project structure

Module/path	Responsibility
app/	Navigation, application setup, theme and localization.
core/designsystem/	Colors, typography, spacing, reusable components and mockup-matched layouts.
core/model/	Currency, expense, group, friend and payment domain models.
core/data/	Firestore repositories, Storage, authentication and offline cache.
feature/auth/	Google/email login and account recovery.
feature/dashboard/	Balances, recent splits and monthly summary.
feature/friends/	Profiles, friend requests, groups and privacy.
feature/map/	Map, location permissions and shared places.
feature/expense/	Create/edit split, calculations and detail.
feature/history/	Monthly analytics, filters and progress.
feature/settings/	Themes, language, currency, export and account controls.
functions/	Trusted validation, payment updates and notifications.

12. Acceptance checklist

- All five screens match the approved ocean-blue mockup in hierarchy, colors, spacing and typography.
- Google and email authentication work with Firebase; no hard-coded demo accounts.
- Friends/groups and consent-based profile details/photos work across devices.
- All split methods reconcile exactly after rounding; taxes, fees, extras and discounts calculate correctly.
- Paid/partial/unpaid/overpaid states derive from real payment ledger entries.
- Dashboard and monthly progress update in real time.
- NPR, INR, PKR and additional currencies format correctly; transaction currency is preserved.
- English default, Nepali and Hindi translations are complete for the launch feature set.
- Map is permission-based and private by default; list view works without location permission.
- PDF/TXT/CSV exports work for selected scopes and preserve Unicode/currency codes.
- Themes, language, currency and profile settings persist.
- Security Rules prevent unauthorized access. Loading, empty, offline, permission-denied and error states are tested.

13. Delivery phases

Phase	Deliverables
1 — Foundation	Firebase setup, Auth, schema, rules, architecture, design system and localization.
2 — Core UI	Login, profile, friends/groups, dashboard and create-split flow.
3 — Ledger	Expense details, split methods, payments, unpaid states, live updates and notifications.
4 — Regional	NPR/INR/PKR and additional currencies; English/Nepali/Hindi.
5 — Map & reports	Privacy-aware map, monthly analytics, progress and Settings exports.

Phase	Deliverables
6 — Release	Security review, emulator tests, device-size QA, accessibility and production deployment.

14. Developer notes

• Build the design system first; compare Compose screens to the approved reference at matching aspect ratios and multiple device sizes.

• Use responsive layouts, safe insets, scrollable content, scalable typography and accessible touch targets.

• Use a money type carrying ISO currency code and integer minor units; do not pass unlabelled Double values.

• Keep financial calculations in tested domain functions and repeat critical validation on the backend.

• Use Firebase Emulator Suite for Auth, Firestore and Functions integration tests.

• Keep development fixtures in debug-only builds; never ship them as live data.

• Do not claim payment processing is integrated unless a regulated payment provider and settlement flow are actually implemented. Manual payment tracking is a separate feature.

Final design constraint: the approved mockup remains the source of truth. Add functionality through secondary pages, bottom sheets and Settings—not by overcrowding the five primary screens.